Pairing Proofs and Polynomial Ring Toolkit

Anonymous Submission

Abstract. Elliptic curve pairings are fundamental to modern zero-knowledge proof systems, including Groth16, PLONK, and KZG polynomial commitments. In resource-constrained execution environments (Ethereum VM, BitVM, blockchain smart contracts) where computation is metered by gas fees, and in arithmetized computation systems (R1CS, Plonkish, Cairo AIR) where each computational step imposes significant prover overhead, the cost of pairing computation remains a practical bottleneck. We present a public-coin interactive proof system that consolidates the Miller loop iterations and final exponentiation into a single polynomial relation. The relation can then be verified as polynomial identities greatly reducing the pairing costs.
We present a full implementation in gnark [Cona]—the industry-standard framework for arithmetized circuit development—as a reusable polynomial ring toolkit in the `emulated` package. Benchmarked on a 3-pairing product check—the pairing workload of a Groth16 verification—our gnark implementation achieves **over 32% reduction in constraints**, **over 50% reduction in committed elements**, and reductions in both compile-time and solver memory for both BN254 and BLS12–381 curves, enabling practical pairing verification in recursive proof systems and resource-constrained environments.

Keywords: Elliptic curve pairings Zero-knowledge proofs · Polynomial identity testing · Blockchain verification · Recursive proofs.

1 Introduction

Blockchain technology continues to mature with improving scalability and infrastructure. However, privacy remains a significant barrier to entry for many users and applications. Zero-knowledge proofs (ZKPs) have emerged as a powerful solution to this challenge.

1.1 Motivation

Elliptic curve pairings represent fundamental building blocks for numerous cryptographic systems including Groth16[Gro16], PLONK[GWC19], BLS signature schemes[BGW+22], and KZG polynomial commitment schemes[KZG10].

However, **pairings are computationally intensive**. This creates critical bottlenecks in two important deployment contexts:

1. **Resource-constrained virtual machines** (Ethereum VM, BitVM, ZKVMs): Computation is metered — each operation incurs gas costs. A single pairing verification requiring thousands of field operations becomes economically prohibitive during peak network congestion.

2. **Arithmetized computation systems** (R1CS [GGPR13], Plonkish [GWC19], Cairo AIR [GPR21]): Each operation must be represented algebraically, adding significant prover overhead beyond native execution costs.

In such contexts, an efficient proof system for pairing correctness can replace expensive native execution inside the constrained environment.

1.2 Our Contribution

We present a public-coin interactive proof system for verifying pairing computations that consolidates verification into unified polynomial relationships under the Schwartz-Zippel lemma. By treating complete Miller loop iterations as single polynomial relationships rather than sequences of individual extension field operations [Fel23], our approach achieves:

- **35% reduction** in SCS constraints (1,592,440 → 1,035,550 for a 3-pairing BN254 Groth16 verification check; 33% on BLS12–381)
- **53% reduction** in committed elements (571,125 → 269,616; 54.5% on BLS12–381)
- up to **∼39% reduction** in compile-time memory and up to **48%** in solver memory

Our key insight is that instead of verifying individual field operations separately, we can verify entire Miller loop iterations as single polynomial identities. This consolidation dramatically reduces both the number of modular operations required and the communication overhead from intermediate variable commitment.

As a reusable artefact of independent interest, we also contribute a **polynomial ring toolkit** for gnark's emulated arithmetic package. The toolkit packages the two-round interactive proof system into a clean API—a single `NewPolyRingCheck` call registers a modulus, `MulPolyRings` submits multiplications with deferred verification, and `performDeferredRingChecks` closes the circuit with two `api.Commit` challenges. Atop this, the `PolyRingAccumulator` provides a higher-level interface for building product chains: factors are enqueued via `Mul`, squarings via `Sqr`, and the accumulated product is evaluated lazily through `Eval`, with optional automatic degree-bounding to keep intermediate products manageable.

2 Background

2.1 Pairing Computation

An elliptic curve pairing is a non-degenerate bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$, where $\mathbb{G}_1$ and $\mathbb{G}_2$ are groups of points on elliptic curves over finite fields, and $\mathbb{G}_T$ is a multiplicative group in an extension field. For practical cryptographic applications, we require that this map be efficiently computable. We will focus on the optimal ate pairing [BGM+10] over Barreto-Naehrig (BN) curves, which is widely used due to its efficiency. We will focus on the Ethereum's BN254 curve, defined over a 254-bit prime field $\mathbb{F}_q$, with groups $\mathbb{G}_1, \mathbb{G}_2$ of order r and target group $\mathbb{G}_T$ contained in the 12th extension field $\mathbb{F}_{q^{12}}$. We also benchmark BLS12–381 (Table 7) for higher security requirements.

The computation of an optimal ate pairing—the focus of our implementation—consists of two primary phases:

1. **Miller loop**: The core iterative computation that evaluates line functions at pairs of points from $\mathbb{G}_1$ and $\mathbb{G}_2$, accumulating intermediate results to produce a value in $\mathbb{F}_{q^{12}}$.
2. **Final Exponentiation**: This step raises the Miller loop output to the power $(q^{12} - 1)/r$, where r is the order of the groups, ensuring the result lies in the correct subgroup $\mathbb{G}_T$.

While both phases present optimization opportunities, the constraint-based verification of these computations introduces unique challenges and possibilities that differ significantly from native implementations.

Algorithm 1 Optimal Ate pairing over Barreto-Naehrig curves [BGM+10]

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2$
Output: $a_{opt}(Q, P)$

Write $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1, 0, 1\}$ and $s_L = 1$
$T \leftarrow Q, f \leftarrow 1$
for $i = L - 2$ to 0 **do**
    $f \leftarrow f^2 \cdot l_{T,T}(P)$
    $T \leftarrow [2]T$
    if $s_i = 1$ **then**
        $f \leftarrow f \cdot l_{T,Q}(P)$
        $T \leftarrow T + Q$
    end if
    if $s_i = -1$ **then**
        $f \leftarrow f \cdot l_{T,-Q}(P)$
        $T \leftarrow T - Q$
    end if
end for
$Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$
$f \leftarrow f \cdot l_{T,Q_1}(P), T \leftarrow T + Q_1$
$f \leftarrow f \cdot l_{T,-Q_2}(P), T \leftarrow T - Q_2$
$f \leftarrow f^{(p^{12}-1)/r}$
return f

2.2 Schwartz-Zippel lemma

The Schwartz-Zippel lemma (or more precisely, the Demillo-Lipton-Schwartz-Zippel lemma [DL78, Sch79, Zip79, Sch80]) provides a probabilistic method for testing polynomial identities. The lemma states that for a non-zero polynomial $P(x_1, x_2, \ldots, x_n)$ of total degree d over a finite field $\mathbb{F}$, when evaluated at uniformly random field elements, the probability that P evaluates to zero is at most $d/|\mathbb{F}|$.

This probabilistic bound becomes negligible for cryptographically sized fields where $d \ll |\mathbb{F}|$, making it practical to verify polynomial identities by evaluation instead of coefficient-wise operations.

2.3 Computation Models

We implement and benchmark our toolkit in gnark [Cona], the industry-standard Go library for arithmetized circuit development, targeting both SCS (Sparse Constraint System, the native Plonkish backend) and R1CS constraint systems. We map each field multiplication to a constraint (denoted by **C**) to maintain compatibility with the R1CS model used in established literature. In R1CS [Gro16], prover complexity is dominated by multiplication gates while additions are free. To synchronize our theoretical analysis with the benchmarks provided by El Housni [Hou23], we define our cost metric based on field multiplications.

2.3.1 R1CS Cost Model

We measure computational cost in terms of *constraints* (denoted by **C**), representing the fundamental unit of work in Rank-1 constraint systems. One constraint corresponds to one multiplication gate.

Basic Field Operations in $\mathbb{F}_q$:

- **Addition/Subtraction:** Free

- **Multiplication:** One constraint (1C)
- **Division/Inversion:** One multiplication + one equality check

Commitment Overhead: Beyond constraint costs, we account for *commitment overhead*—the number of field elements the prover must commit to (via Fiat-Shamir hashing in non-interactive proofs). For blockchain deployments, this overhead directly impacts transaction costs and is often the dominant factor.

2.3.2 Cost Model Comparison

Operations that are traditionally expensive (inversions, divisions) become cheap in constraint systems when computed via witnesses. This inverts traditional optimization priorities: we can afford inversions to save multiplications, enabling affine coordinate systems and other techniques impractical in native implementations.

Recognizing and exploiting these unique computational properties is essential for unlocking hidden performance potential. Youssef El Housni's pioneering work **Pairings in Rank-1 Constraint Systems** [Hou23] represents, to our knowledge, the first systematic exploration of these optimization opportunities, providing a breakthrough in the practicality of recursive proving by thoughtfully leveraging the distinctive cost model inherent to constraint-based systems.

3 Related Work

The landscape of efficient pairing verification in constraint systems has evolved through three key contributions that our work directly builds upon.

1. El Housni's **Pairings in Rank-1 Constraint Systems** [Hou23] established the theoretical framework for optimizing pairings in arithmetized environments, introducing efficient Miller loop formulas in affine coordinates and sparse line function representations. This work inverts the usual cost model: inversions become cheap (2C) while multiplications remain the dominant cost.

2. Feltroidprime [Fel23] introduced polynomial identity testing for $\mathbb{F}_{q^{12}}$ arithmetic via the Schwartz-Zippel lemma. Each multiplication $A \cdot B$ is expressed as $A(x) \cdot B(x) = Q(x) \cdot P_{12}(x) + R(x)$ and verified at a single random point, with the quotients Q batched via random linear combination. Our work applies the same polynomial framework but at a coarser granularity: entire Miller loop steps rather than individual multiplications, eliminating intermediate remainder commitments entirely. Concretely, a 3-pair zero-bit Miller loop step requires four chained relations and 48 $\mathbb{F}_q$ commitments under [Fel23], versus one relation and 12 commitments in our scheme (Section 4.1).

3. Novakovic and Eagen's **On Proving Pairings** [NE24] showed that final exponentiation can be replaced by a residue witness check, reducing final exponentiation cost by over 85% for BN254. We integrate their residue witness c directly into our polynomial step equations (Section 4.2.2), unifying the Miller loop and final exponentiation into the same polynomial verification framework.

The **Garaga** library [FE] deployed an early Cairo-specific proof-of-concept of the consoli-

dation technique in 2024, adapted from a preceding technical write-up [Ano24]. Garaga demonstrates practicality in production on Starknet but provides no formal treatment. This paper supplies the missing foundations—the operation polynomials of Section 4.2, the batched two-round protocol and its soundness analysis in Section 5—and evaluates the technique systematically against the strongest available baseline: gnark's emulated pairing stack, which incorporates the optimizations of both [Hou23] and [NE24].

4 Pairing Operations as Polynomials

Algorithm 2 gives the pairing computation combining El Housni's R1CS optimizations [Hou23] with Novakovic-Eagen's residue witness technique [NE24]. The 27th root of unity W scales the Miller loop output f to be a cubic residue (see [NE24] §4.3); if f is not already a cubic residue, multiplying by W^w, $w \in \{1, 2\}$ corrects this.

Our contribution is to express each step of this algorithm as a single multi-operand polynomial product verified via the Schwartz-Zippel lemma, rather than as a sequence of individual binary multiplications as in [Fel23].

4.1 Comparative Analysis

We compare the two schemes for a 3-pair Miller loop zero-bit step (the most common case in BN254's NAF[DHM04] representation of $s = 6t + 2$). Here $F \in \mathbb{F}_{q^{12}}$ is the accumulator, $L_i \in \text{Sparse}_{01379}$ are the three doubling line functions, $R \in \mathbb{F}_{q^{12}}$ is the updated accumulator, and Q is the quotient polynomial. Sparse_{01379} denotes the subspace of $\mathbb{F}_{q^{12}}$ elements whose only nonzero coefficients are at degrees $0, 1, 3, 7, 9$—the form that BN254 line functions naturally take [Hou23], reducing their evaluation cost from 12C to 4C. All compared schemes[Hou23]—, [Fel23], and ours—exploit this sparseness of the Miller line functions; the corresponding savings are included in every constraint count below.

4.1.1 Previous scheme

Adapting [Fel23] to lines 6–7 of Algorithm 2 for a 3-pair Miller loop gives:

$$F(x) \cdot F(x) \stackrel{?}{=} Q_1(x) \cdot P_{12}(x) + T_1(x)$$
$$T_1(x) \cdot L_1(x) \stackrel{?}{=} Q_2(x) \cdot P_{12}(x) + T_2(x)$$
$$T_2(x) \cdot L_2(x) \stackrel{?}{=} Q_3(x) \cdot P_{12}(x) + T_3(x)$$
$$T_3(x) \cdot L_3(x) \stackrel{?}{=} Q(x) \cdot P_{12}(x) + R(x)$$

Evaluating F, T_1–T_3, R costs 12C each and L_1–L_3 cost 4C each; all four remainder polynomials require commitment. **Total: 76C and 48 $\mathbb{F}_q$ commitments** (4×12).

4.1.2 Our scheme

Our scheme consolidates lines 6–7 into a single polynomial identity:

$$F(x) \cdot F(x) \cdot L_1(x) \cdot L_2(x) \cdot L_3(x) \stackrel{?}{=} Q(x) \cdot P_{12}(x) + R(x)$$

By eliminating intermediate T_* polynomials, only R requires commitment. **Total: 41C and 12 $\mathbb{F}_q$ commitments** ($2 \times 12\text{C} + 3 \times 4\text{C} + 5\text{C}$).

This results in **46% fewer constraints** and a **75% reduction in commitments**. When verifying ZK proofs on-chain, commitment savings are often more critical than constraint savings, due to the calldata costs associated with transmitting polynomial hints.

Algorithm 2 Optimal Ate pairing with Residue witness check

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, c \in \mathbb{F}_{q^{12}}, w \in \{0, 1, 2\}$
Internal $W, W^2 \in \mathbb{F}_{q^{12}}$ ▷ W is 27th root of unity
Output: $a_{opt}(Q, P)$

Write $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1, 0, 1\}$ and $s_L = 1$
$c^{-1} \leftarrow 1/c$ ▷ Inverse Residue witness
$f \leftarrow c^{-1}$ ▷ Initialize with inverse residue witness
$T \leftarrow Q$ ▷ T_i for each pair for multi-pairing
Every operation involving T, P or Q is repeated for T_i, P_i and Q_i for multi-pairing.

for $i = L - 2$ to 0 **do**
  $f \leftarrow f^2$
  $f \leftarrow f \cdot l_{T,T}(P)$ ▷ Repeat for T_i, P_i
  $T \leftarrow [2]T$ ▷ Repeat for T_i
  if $s_i = 1$ **then**
    $f \leftarrow f \cdot c^{-1}$ ▷ Residue witness
    $f \leftarrow f \cdot l_{T,Q}(P)$ ▷ Repeat for T_i, Q_i, P_i
    $T \leftarrow T + Q$ ▷ Repeat for T_i, Q_i
  else if $s_i = -1$ **then**
    $f \leftarrow f \cdot c$ ▷ Residue witness
    $f \leftarrow f \cdot l_{T,-Q}(P)$ ▷ Repeat for $T_i, -Q_i, P_i$
    $T \leftarrow T - Q$ ▷ Repeat for $T_i, -Q_i$
  end if
end for

$Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$ ▷ Repeat for Q_{1i}, T_i
$f \leftarrow f \cdot l_{T,Q_1}(P)$ ▷ Repeat for Q_{1i}, T_i, P_i
$T \leftarrow T + Q_1$ ▷ Repeat for Q_{1i}, T_i
$f \leftarrow f \cdot l_{T,-Q_2(Q)}(P)$ ▷ Repeat for Q_i, T_i, P_i
if $w = 1$ **then**
  $f \leftarrow f \cdot W$
else if $w = 2$ **then**
  $f \leftarrow f \cdot W^2$
end if
$f \leftarrow f \cdot c^{-q} \cdot c^{q^2} \cdot c^{-q^3}$ ▷ Uses Frobenius maps, Negative exponents use c^{-1}

return f

4.1.3 Performance Comparison

Table 1 compares the computational costs and commitment overhead between the schemes for a single 0-bit Miller loop operation with 3 pairs.

4.2 Operation Polynomials

Our implementation uses different polynomials for different bits in the Miller loop. Let us walk through a 3-pairing setup, the configuration used for Groth16 verification (Section 7.2). We will describe the polynomials for zero bits, non-zero bits and correction step.

Table 1: 3-Pair Miller loop 0-bit constraints comparison

Scheme	Constraints	$\mathbb{F}_q$ Commitments	Improvement
[Hou23]	132C	—	Baseline
[Fel23]	76C*	48	42%
Our scheme	**41C***	**12**	**69%**

[Hou23]: 1 square (36C) + 3 sparse mul (3×30C) = 132C
[Fel23]: 5 evals (5×12C) + 3 line evals (3×4C) + 4 muls (4×1C) = 76C
Our scheme: 2 evals (2×12C) + 3 line evals (3×4C) + 5C (muls + RLC) = 41C
*Costs exclude Fiat-Shamir commitment overhead

4.2.1 Miller loop: Zero Bit Equation

As seen in 4.1.2 for zero bit operations (line 6 and 7 with three $l_{T,T}(P)$ lines in Algorithm 2), the verification equation is:

$$F^2(x) \cdot L_1(x) \cdot L_2(x) \cdot L_3(x) = R(x) + Q(x) \cdot P_{12}(x) \tag{1}$$

Table 2: Zero bit operation polynomial components

Polynomial	Description	Degree	Coefficients
$F \in \mathbb{F}_{q^{12}}$	Miller loop accumulator	$11 \times 2 = 22$	12
$L_1, L_2, L_3 \in \text{Sparse}_{01379}$	The line functions	$9 \times 3 = 27$	$4 \times 3 = 12$
$R \in \mathbb{F}_{q^{12}}$	Remainder, The result	11	12
Total constraints	36C Evals, 4C Muls and 1C RLC		**41**
Q *(Amortized See 5.1)*	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	38	39

4.2.2 Miller loop: Non-Zero Bit Equation

For non-zero bit operations(lines 10, 11, 14, 15 with three pairs of lines in Algorithm 2), the Miller loop performs both doubling and addition steps. So for each pair we have two Sparse_{01379} lines. The verification equation becomes:

$$F^2(x) \cdot W(x) \cdot L_{\text{d1}}(x) \cdot L_{\text{d2}}(x) \cdot L_{\text{d3}}(x) \cdot L_{\text{a1}}(x) \cdot L_{\text{a2}}(x) \cdot L_{\text{a3}}(x) = R(x) + Q(x) \cdot P_{12}(x) \tag{2}$$

Table 3: Non-zero bit operation polynomial components

Polynomial	Description	Degree	Coefficients
$F \in \mathbb{F}_{q^{12}}$	Miller loop accumulator	$11 \times 2 = 22$	12
$L_{\text{d1}}, L_{\text{d2}}, L_{\text{d3}} \in \text{Sparse}_{01379}$	Doubling line functions	$9 \times 3 = 27$	$4 \times 3 = 12$
$L_{\text{a1}}, L_{\text{a2}}, L_{\text{a3}} \in \text{Sparse}_{01379}$	Addition line functions	$9 \times 3 = 27$	$4 \times 3 = 12$
W or $W^{-1} \in \mathbb{F}_{q^{12}}$	Residue witness	11	12
$R \in \mathbb{F}_{q^{12}}$	Remainder, The result	11	12
Total constraints	60C Evals, 8C Muls and 1C RLC		**69**
Q *(Amortized See 5.1)*	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	76	77

The residue witness W encapsulates the equivalence class relationship established by Novakovic and Eagen's final exponentiation elimination. The inverse of W is used for negative bits, this just changes the coefficients the equation remains the same.

4.2.3 Miller loop: Frobenius mapping Equation

The correction step finalizes the Miller loop by incorporating the Frobenius endomorphism corrections (lines 20 and 22 in Algorithm 2). Again, we have two Sparse_{01379} lines for each pair, for π_p and $-\pi_p^2$ mappings, without any squaring of the accumulator. The verification equation becomes:

$$F(x) \cdot L_{1\pi}(x) \cdot L_{2\pi}(x) \cdot L_{3\pi}(x) \cdot L_{1\pi^2}(x) \cdot L_{2\pi^2}(x) \cdot L_{3\pi^2}(x) = R(x) + Q(x) \cdot P_{12}(x) \quad (3)$$

Table 4: Correction step polynomial components

Polynomial	**Description**	**Degree**	**Coefficients**
$F \in \mathbb{F}_{q^{12}}$	Miller loop accumulator	11	12
$L_{1\pi}, L_{2\pi}, L_{3\pi} \in \text{Sparse}_{01379}$	π_p correction lines	$9 \times 3 = 27$	$4 \times 3 = 12$
$L_{1\pi^2}, L_{2\pi^2}, L_{3\pi^2} \in \text{Sparse}_{01379}$	$-\pi_p^2$ correction lines	$9 \times 3 = 27$	$4 \times 3 = 12$
$R \in \mathbb{F}_{q^{12}}$	Remainder, The result	11	12
Total constraints	48C Evals, 6C Muls and 1C RLC		**55**
Q *(Amortized See 5.1)*	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	54	55

4.2.4 Final Exponentiation Equation

The final exponentiation check is handled as described in [NE24]. The equation handles the cubic scale factor multiplication, and **Frobenius component** $(q - q^2 + q^3)$ of the final exponentiation (lines 24, 26 and 28 in Algorithm 2). The verification equation looks like:

$$F(x) \cdot c^{-q}(x) \cdot c^{q^2}(x) \cdot c^{-q^3}(x) \cdot W^w(x) = 1 + Q(x) \cdot P_{12}(x) \quad (4)$$

where $w \in \{0, 1, 2\}$ determines the cubic root of unity scaling factor, and the Frobenius mappings are computed using the residue witness c and its inverse and provided as input. Cubic root is sparse element with just 2 coefficients the degree ranging from 8 to 10 because of positioning of the coefficients. Our R is now 1 for a successful proof verification.

Table 5: Final exponentiation polynomial components

Polynomial	**Description**	**Degree**	**Coefficients**
$F \in \mathbb{F}_{q^{12}}$	Final Miller loop result	11	12
$c^{-q} \in \mathbb{F}_{q^{12}}$	Residue witness Frobenius $-q$	11	12
$c^{q^2} \in \mathbb{F}_{q^{12}}$	Residue witness Frobenius q^2	11	12
$c^{-q^3} \in \mathbb{F}_{q^{12}}$	Residue witness Frobenius $-q^3$	11	12
$W^w \in \mathbb{F}_{q^{12}}$	Cubic scale factor	10	2
$R = 1$	Unity (constant)	0	1
Total constraints	51C Evals, 4C Muls and 1C RLC		**56**
Q *(Amortized See 5.1)*	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	43	44

With this we have all the polynomials we need for verifying entire pairing operation. Next we will look into high level overview of pairing verification via an interactive oracle proof.

4.3 Security

Each verification equation takes the form $P = \prod_i A_i(x) - Q(x) \cdot P_{12}(x) - R(x)$ where $\deg(P) < 2^8$.[1] By the Schwartz-Zippel lemma (Section 2.2):

[1] For k-pair pairings, $\deg(P) \approx 88 + 18(k-3)$, well below 2^8 for all practical k.

Soundness: $\delta_s \leq d/|\mathbb{F}_q| < 2^8/2^{254} = 2^{-246}$ for BN254.

Completeness: An honest prover always produces $P(x) = 0$ identically, so the verifier always accepts. Euclidean division guarantees unique Q, R with $\deg(R) < \deg(P_{12}) = 12$.

5 Full Pairing Proof with Quotient Batching

We now combine the polynomial equations from Section 4.2 into a complete two-round public-coin interactive proof, further reducing verification cost by batching all quotients into a single accumulated polynomial.

5.1 Batching Polynomial Identity Testing

All n polynomial equations from Section 4.2 are batched into a single verification. The prover commits to remainders $\{R_i\}_{i=0}^{n-1}$ and receives challenge z. Using z, the prover forms the accumulated quotient $Q_{\text{acc}} = \sum_{i=0}^{n-1} z^i \cdot Q_i$. The verifier then samples a second challenge point c and checks:

$$\sum_{i=0}^{n-1} z^i \cdot [A_i(c) \cdot B_i(c) \cdots X_i(c) - R_i(c)] = Q_{\text{acc}}(c) \cdot P_{12}(c) \tag{5}$$

This reduces n polynomial verifications (degrees 38–76, see Tables 2–5) to a single evaluation of a degree-76 polynomial.

5.2 Prover's Algorithms

The prover computes quotients Q_i and remainders R_i for each equation in Section 4.2. For each step, it calls PolyMulDiv, which multiplies the input polynomials and performs standard Euclidean division by the irreducible P_{12}. After receiving challenge z, the prover forms $Q_{\text{acc}} = \sum_i z^i Q_i$ and sends $\mathcal{R} = \{R_i\}$ and Q_{acc} to the verifier. The full prover procedure is given in Algorithm 3.

5.3 Verifier's Algorithms

The verifier receives $P \in \mathbb{G}_1$, $Q \in \mathbb{G}_2$, residue witness c [NE24], the W (27th root of unity), the remainder array $\mathcal{R}$, and Q_{acc} from the prover. Using challenge z from the first round and a freshly sampled x, it mirrors the prover's pairing loop while evaluating each polynomial at x, accumulating the RLC e_{acc}. The final check is Equation 5. The full verifier is given in Algorithm 4.

Algorithm 3 Pairing Prover: Preparing polynomials

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, c \in \mathbb{F}_{q^{12}}, w \in \{0,1,2\}$
Internal: $W, W^2 \in \mathbb{F}_{q^{12}}$ ▷ W is 27th root of unity
Output: Arrays $\mathcal{Q}, \mathcal{R}$ of quotients and remainders
Write $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1, 0, 1\}$ and $s_L = 1$
$c^{-1} \leftarrow 1/c$ ▷ Inverse Residue witness
$f \leftarrow c^{-1}$ ▷ Initialize with inverse residue witness
$T \leftarrow Q$ ▷ T_i for each pair for multi-pairing
Every operation involving T, P or Q is repeated for T_i, P_i and Q_i for multi-pairing.

function `PolyMulDiv` $(R, P_1, \ldots, P_n \in \mathbb{F}_q[x])$: **return** $Q, R \in \mathbb{F}_q[x]$
s.t. $\prod P_i = Q \cdot P_{12} + R$, $\deg(R) < \deg(P_{12})$

Miller loop
for $i = L - 2$ to 0 **do**
  $L_{\text{dbl}} \leftarrow l_{T,T}(P)$, $T \leftarrow [2]T$
  if $s_i = 0$ **then**
    $Q, R \leftarrow PolyMulDiv(f, f, L_{\text{dbl}})$ ▷ add L_{*i} for each P_i, Q_i
  else if $s_i = 1$ **then**
    $L_{\text{add}} \leftarrow l_{T,Q}(P)$, $T \leftarrow T + Q$
    $Q, R \leftarrow PolyMulDiv(f, f, c^{-1}, L_{\text{dbl}}, L_{\text{add}})$ ▷ add L_{*i} for each P_i, Q_i
  else if $s_i = -1$ **then**
    $L_{\text{add}} \leftarrow l_{T,-Q}(P)$, $T \leftarrow T - Q$
    $Q, R \leftarrow PolyMulDiv(f, f, c, L_{\text{dbl}}, L_{\text{add}})$ ▷ add L_{*i} for each P_i, Q_i
  end if
  $f \leftarrow R$, $\mathcal{Q}.append(Q)$, $\mathcal{R}.append(R)$
end for

$Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$
$L_\pi \leftarrow l_{T,Q_1}(P)$
$T \leftarrow T + Q_1$
$L_{-\pi^2} \leftarrow l_{T,-Q_2}(P)$
$Q, R \leftarrow PolyMulDiv(f, L_\pi, L_{-\pi^2})$ ▷ add L_{*i} for each P_i, Q_i
$f \leftarrow R$, $\mathcal{Q}.append(Q)$, $\mathcal{R}.append(R)$

Final exponentiation: Cubic scaling + Frobenius mappings
if $w = 0$ **then**
  $Q, R \leftarrow PolyMulDiv(f, c^{-q}, c^{q^2}, c^{-q^3})$
else if $w = 1$ **then**
  $Q, R \leftarrow PolyMulDiv(f, W, c^{-q}, c^{q^2}, c^{-q^3})$
else if $w = 2$ **then**
  $Q, R \leftarrow PolyMulDiv(f, W^2, c^{-q}, c^{q^2}, c^{-q^3})$
end if
$\mathcal{Q}.append(Q)$, $\mathcal{R}.append(R)$ ▷ Final $\mathcal{Q}$ and $\mathcal{R}$ accumulation, skip f

return $\mathcal{Q}, \mathcal{R}$

Algorithm 4 Pairing Verifier: Evaluating remainders

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, c \in \mathbb{F}_{q^{12}}, w \in \{0,1,2\}$, $\mathcal{R}$ array of remainders, $Q_{\text{acc}} \in \mathbb{F}_q[x]$
Internal: 27th root of unity, $W, W^2 \in \mathbb{F}_{q^{12}}$, first round challenge, $z \xleftarrow{\$} \mathbb{F}_q$
Output: $pairing(P,Q) \stackrel{?}{=} 1$
Write $s = 6t+2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1,0,1\}$ and $s_L = 1$

$c^{-1} \leftarrow 1/c$ ▷ Inverse Residue witness
$f \leftarrow c^{-1}$ ▷ Initialize with inverse residue witness
$T \leftarrow Q$ ▷ T_i for each pair for multi-pairing
$x \xleftarrow{\$} \mathbb{F}_q$ ▷ Sample evaluation point from field
$e_{\text{acc}} \leftarrow 0$, $a_{\text{rlc}} \leftarrow 1$ ▷ Evaluation accumulator and random linear combination multiplier
Every operation involving T, P or Q is repeated for T_i, P_i and Q_i for multi-pairing.

function `EvalPoly` $(R, P_1, \ldots, P_n \in \mathbb{F}_q[x])$: **return** $(\prod_{i=1}^{n} P_i(z) - R(z) \in \mathbb{F}_q)$

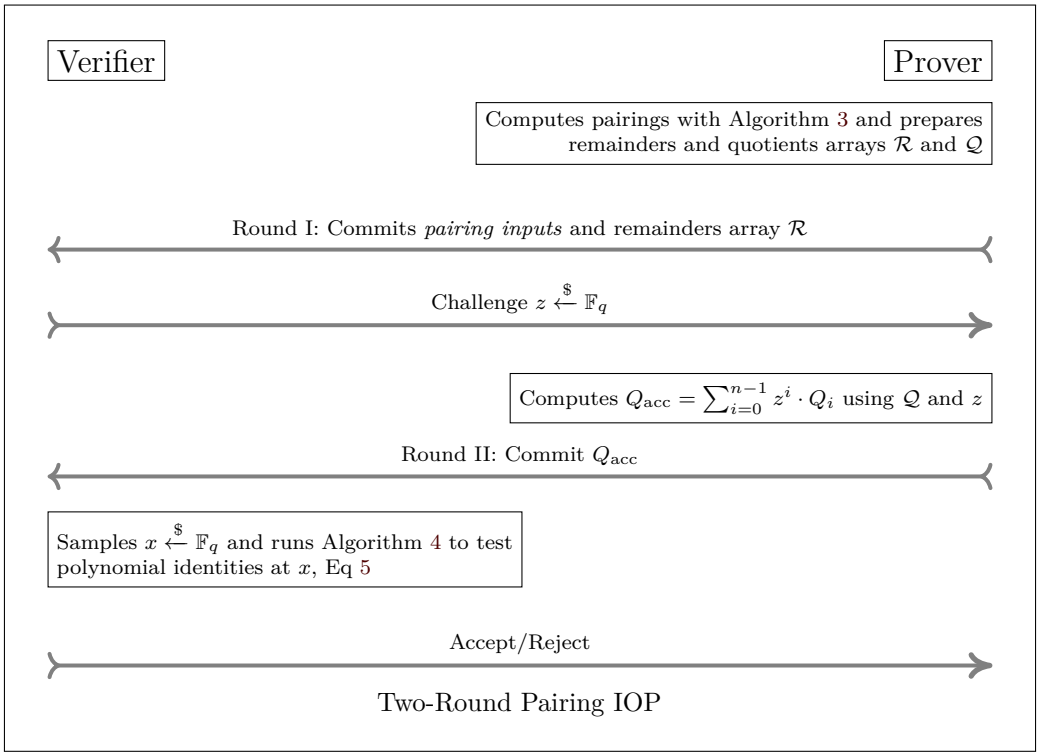
Miller loop
for $i = L-2$ to 0 **do**
  $L_{\text{dbl}} \leftarrow l_{T,T}(P)$, $T \leftarrow [2]T$
  $R \leftarrow \mathcal{R}.pop\_front()$ ▷ Fetch R from prover
  if $s_i = 0$ **then**
    $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \texttt{EvalPoly}(R, x, f, f, L_{\text{dbl}})$ ▷ add L_{*i} for each P_i, Q_i
  else if $s_i = 1$ **then**
    $L_{\text{add}} \leftarrow l_{T,Q}(P)$, $T \leftarrow T + Q$
    $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \texttt{EvalPoly}(R, x, f, f, c^{-1}, L_{\text{dbl}}, L_{\text{add}})$ ▷ add L_{*i} for each P_i, Q_i
  else if $s_i = -1$ **then**
    $L_{\text{add}} \leftarrow l_{T,-Q}(P)$, $T \leftarrow T - Q$
    $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \texttt{EvalPoly}(R, x, f, f, c, L_{\text{dbl}}, L_{\text{add}})$ ▷ add L_{*i} for each P_i, Q_i
  end if
  $f \leftarrow R$, $a_{\text{rlc}} \leftarrow a_{\text{rlc}} \cdot z$ ▷ Update random linear combination factor
end for

$R \leftarrow \mathcal{R}.pop\_front()$ ▷ Fetch R from prover
$Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$
$L_\pi \leftarrow l_{T,Q_1}(P)$
$T \leftarrow T + Q_1$
$L_{-\pi^2} \leftarrow l_{T,-Q_2}(P)$
$e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \texttt{EvalPoly}(R, x, f, L_\pi, L_{-\pi^2})$ ▷ add L_{*i} for each P_i, Q_i
$f \leftarrow R$, $a_{\text{rlc}} \leftarrow a_{\text{rlc}} \cdot z$ ▷ Update random linear combination factor

Final exponentiation: Cubic scaling + Frobenius mappings
$R \leftarrow \mathcal{R}.pop\_front()$ ▷ Fetch R from prover
if $w = 0$ **then**
  $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \texttt{EvalPoly}(R, x, f, c^{-q}, c^{q^2}, c^{-q^3})$
else if $w = 1$ **then**
  $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \texttt{EvalPoly}(R, x, f, W, c^{-q}, c^{q^2}, c^{-q^3})$
else if $w = 2$ **then**
  $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \texttt{EvalPoly}(R, x, f, W^2, c^{-q}, c^{q^2}, c^{-q^3})$
end if

return $Q_{\text{acc}}(x) \stackrel{?}{=} e_{\text{acc}} \wedge R \stackrel{?}{=} 1$ ▷ Polynomial identity and expected pairing output check

5.4 Interactive Proof



5.5 Fiat-Shamir Transformation

Applying the Fiat-Shamir transformation [FS87] with random oracle H to the two-round protocol of Section 5.4 gives a non-interactive proof with knowledge soundness $\delta_s + 3(m^2 + 1)2^{-\lambda}$ [BSCS16], where $m = 2$ is the number of oracle queries (one per round) and λ is the challenge bit-length; Section 5.6.2 instantiates this bound.

5.5.1 Non-Interactive Proof

The prover runs Algorithm 3 to obtain $(\mathcal{Q}, \mathcal{R})$, derives the challenge

$$z = H(\textit{pairing inputs}, c, W^w, \mathcal{R})$$

from the transcript instead of receiving it, computes $Q_{\text{acc}} = \sum_{i=0}^{n-1} z^i \cdot Q_i$, and outputs the proof $(\textit{pairing inputs}, c, W^w, \mathcal{R}, Q_{\text{acc}})$. The verifier recomputes z, derives the evaluation point $x = H(z, Q_{\text{acc}})$, and runs Algorithm 4, testing the polynomial identities via Equation (5).

The soundness bound of Theorem 1 carries over because every prover-chosen value entering Equation (5) is absorbed before the challenge it must not depend on: $\mathcal{R}$ and the hinted witnesses c, W^w before z, and Q_{acc} with z chained into the second query before x.

5.5.2 Instantiating the Random Oracle

The protocol is agnostic to how H is realised; the cost of absorbing one $\mathbb{F}_q$ element into

the transcript varies across execution environments. We therefore qualify the committed element counts of Section 4.1 and Table 6 for our two instantiations.

CairoVM (Poseidon). Prover-supplied hints are passed as transaction calldata and absorbed into a running Poseidon [GKR+21] sponge; $\mathbb{F}_q$ elements are represented by 96-bit limbs, with two limbs packed into each `Felt252` absorption. Both hashing cost and calldata therefore scale linearly with the number of committed elements, which is why the COMMITS column of Table 6 is reported alongside modular operations: halving the committed elements roughly halves the transcript-related share of VM steps and the hint calldata.

Faster Hash Verification [BSB23] in gnark. Belling et al. [BSB23] provide commitment schemes that allow the prover to compute commitment natively (outside the circuit) and attach an Argument of Knowledge to prove correctness. The scheme is described for both Groth16 and PLONK ([BSB23] Section 6.1 and 6.2). Belling later extends this to support multiple commitment for multiple rounds for gnark in [Bel].

This is made available via `api.Commit` calls in gnark. The costs are backend-specific.

Groth16 in gnark. Each commitment costs two $\mathbb{G}_1$ MSMs of size equal to the number of committed wires ([BSB23] Section 6.1). Proof size grows by one $\mathbb{G}_1$ point per commitment, plus a single knowledge proof folded across all commitments. Verification gains one two-pairing product check (the knowledge proof) and one extra scalar multiplication in the public-input MSM per commitment. None of this touches the constraint count, since the commitment lives entirely outside the arithmetization.

PLONK in gnark. The overhead shifts almost entirely into the constraint system instead. Each committed wire adds one constraint ([BSB23] Section 6.2), so cost scales with the number of committed *wires*. Proof size grows by one $\mathbb{G}_1$ element per commitment. Verification pays no extra pairing: the committed polynomial's evaluation is folded into the batched KZG opening that PLONK already performs, so the marginal verifier cost is a handful of field operations.

5.6 Security Analysis

Throughout this section, $z, x \in \mathbb{F}_q$ denote field element challenges (scalars), while polynomials are elements of $\mathbb{F}_q[X]$ with X as the formal indeterminate. In particular, z^j is a scalar coefficient, not a polynomial term.

5.6.1 Algebraic Intuition

Before the formal soundness proof it is instructive to consider why an incorrect remainder is difficult to hide. Suppose a malicious prover corrupts exactly one remainder: $R'_j = R_j + \Delta_j$ with $\Delta_j \in \mathbb{F}_q[X]$, $\Delta_j \neq 0$, $\deg(\Delta_j) \leq 11$. Since z is obtained *after* the commitment to $\mathcal{R}$, the prover cannot adjust any other R_k to compensate for Δ_j. To successfully deceive the verifier, the prover must find a valid forged quotient $Q'_{\text{acc}} \in \mathbb{F}_q[X]$ satisfying

$$\sum_{i=0}^{n-1} z^i (A_i - R'_i) = Q'_{\text{acc}} \cdot P_{12}$$

All $R'_i = R_i$ except for $R'_j = R_j + \Delta_j$. Substituting $R'_j = R_j + \Delta_j$ gives

$$Q'_{\text{acc}} \cdot P_{12} = Q_{\text{acc}} \cdot P_{12} - z^j \Delta_j(x)$$

$$Q'_{\text{acc}} = Q_{\text{acc}} - \frac{z^j \Delta_j(x)}{P_{12}(x)}.$$

Here $z^j \in \mathbb{F}_q$ is a nonzero scalar (with overwhelming probability over the choice of z) and $\Delta_j(x) \in \mathbb{F}_q[X]$ is a nonzero polynomial of degree at most 11. Since P_{12} is irreducible of degree 12 and $\deg(z^j \Delta_j) \leq 11 < 12 = \deg(P_{12})$, the polynomial P_{12} does not divide $z^j \Delta_j$ in $\mathbb{F}_q[X]$; hence $z^j \Delta_j / P_{12}$ is not a polynomial. Consequently no valid $Q'_{\text{acc}} \in \mathbb{F}_q[X]$ can satisfy the equation for this single-corruption case.

The same degree argument extends immediately to an adversary who corrupts *multiple* remainders: for any scalar z, the weighted sum $\sum_i z^i \Delta_i(x)$ is still a polynomial in x of degree at most 11 (since each z^i is a scalar and each $\deg(\Delta_i) \leq 11$), so it cannot be divisible by P_{12} of degree 12 unless it is the zero polynomial, which requires every $\Delta_i = 0$. The formal soundness proof below re-derives this uniformly via Schwartz-Zippel.

5.6.2 Soundness

Theorem 1 (Soundness)**.** *Let n be the number of polynomial equations batched via a random linear combination into the bivariate polynomial identity:*

$$P_{\text{final}}(z, x) = \sum_{i=0}^{n-1} z^i \cdot \left[\prod_j P_{i,j}(x) - R_i(x) \right] - Q_{\text{acc}}(x) \cdot P_{12}(x) = 0$$

Let the maximum degree in x of the polynomial products be $d_x < 2^8$. For a prover committing to incorrect remainders, the probability of successful forgery δ_s over randomly chosen $(z, x) \in \mathbb{F}_q^2$ is bounded by:

$$\delta_s \leq \frac{n - 1 + d_x}{|\mathbb{F}_q|}$$

Proof. If the prover commits to an incorrect remainder $R_i(x)$, the evaluation yields a non-zero error polynomial $P_{\text{err}}(z, x)$. The structure of $P_{\text{final}}(z, x)$ establishes the following degree bounds for $P_{\text{err}}(z, x)$:

- **Degree in z:** at most $n - 1$ (due to the z^i weighting).
- **Degree in x:** at most $d_x < 2^8$ (due to the polynomial products).

By the Schwartz-Zippel lemma, the probability that the non-zero polynomial P_{err} evaluates to zero is bounded by the total degree divided by the field size:

$$\Pr[P_{\text{err}}(z, x) = 0] \leq \frac{\deg_z(P_{\text{err}}) + \deg_x(P_{\text{err}})}{|\mathbb{F}_q|} \leq \frac{n - 1 + d_x}{|\mathbb{F}_q|}$$

□

Concrete parameters for BN254: With $n = 67$ equations and $d_x \leq 256$:

$$\delta_s \leq \frac{66 + 256}{2^{254}} < \frac{2^9}{2^{254}} = 2^{-245}.$$

Non-interactive proof via Fiat-Shamir : From Section 5.5, under the random oracle model the knowledge soundness error becomes [BSCS16]:

$$\delta'_s = \delta_s + 3(m^2 + 1) \cdot 2^{-\lambda}$$

where $m = 2$ (one query per round) and $\lambda = 254$ (challenge bit-length), giving

$$\delta'_s \leq 2^{-245} + 15 \cdot 2^{-254} < 2^{-244}.$$

Remark 1 (Correctness of polynomial evaluation). Theorem 1 assumes that the verifier evaluates all polynomials at the challenge point x without arithmetic error. The circuit implementation uses the deferred-reduction inner-product optimisation of Section 6.5 (Section 6.5.1), which accumulates limb-wise products over $\mathbb{Z}$ before a single final reduction. Correctness of this optimisation requires that no intermediate limb value reaches the native field characteristic p. We verify this bound concretely for BN254 in Lemma 1 (Section 6.5.1).

5.6.3 Completeness

For an honest prover executing Algorithm 3 with valid inputs:

1. **Polynomial construction**: At each operation, the prover computes following polynomials:
$$\prod_j P_{i,j} = Q_i \cdot P_{12} + R_i$$
2. **RLC formation**: Given challenge z, the prover forms:
$$Q_{\text{acc}} = \sum_{i=0}^{n-1} z^i \cdot Q_i$$
3. **Verifier's check**: The verifier evaluates at point x:
$$\begin{aligned}
\text{LHS} &= \sum_{i=0}^{n-1} z^i \cdot \left[\prod_j P_{i,j}(x) - R_i(x)\right] \\
&= \sum_{i=0}^{n-1} z^i \cdot Q_i(x) \cdot P_{12}(x) \quad \text{(by construction)} \\
&= P_{12}(x) \cdot \sum_{i=0}^{n-1} z^i \cdot Q_i(x) \\
&= P_{12}(x) \cdot Q_{\text{acc}}(x) = \text{RHS}
\end{aligned}$$

Since the equality holds for any evaluation point x, the protocol satisfies perfect completeness.

6 Polynomial Ring Toolkit

The protocol of Section 5 is packaged as a reusable toolkit in the `emulated` package of gnark [Conb], parameterised by any emulated field $\mathbb{F}_q$ (via `FieldParams` type T) and any modulus polynomial $P \in \mathbb{F}_q[x]$. The toolkit operates in a recursive proof setting where the prover's computations are performed outside the circuit via `hints` (Section 6.2), while the verifier's checks are implemented as constraints inside the circuit.

Refer to our fork at https://anonymous.4open.science/r/tiny-gnark-DDCD for code.

6.1 Ring Representation

A ring element is essentially a coefficient slice `[]*Element[T]` in ascending degree order. A `nil` entry is treated as zero by `InnerProductNoReduce`—the term is skipped entirely—so sparse polynomials carry no wasted constraint variables.

The optional `modEvalFn` of type `EvalFnType[T]` = ([`*Element[T]`]) → `*Element[T]` allows handling small-integer coefficients efficiently by multiplying by the small positive integer first and negating afterwards, rather than multiplying by the full-width field representative $q - c$.

Example: BN254 $\mathbb{F}_{p^{12}}$

$P_{12}(x) = x^{12} - 18x^6 + 82$ has nine zero coefficients (stored as `nil`) and two small non-zero ones. The `modEvalFn` computes $P_{12}(\alpha) =$ `MulConst`$(\alpha^0, 82) +$ `Neg`(`MulConst`$(\alpha^6, 18)) + \alpha^{12}$, keeping intermediate values small instead of multiplying by $q - 18$.

6.2 Hint Functions

Hints allow defining computations outside of a circuit. A hint defines an annotated function in the circuit at compile time. Inputs and outputs are injected at the solving time. We define these two hints to implement the prover's steps in the two-round protocol of Section 5.4.

`polyRingMulHint` implements PolyMulDiv: it multiplies the input polynomials, divides by P to obtain (Q, R).

`quotientsRLCHint` computes the accumulated quotient $Q_{\text{acc}} = \sum_i z^i \cdot Q_i \bmod q$ coefficient-wise, corresponding to the prover's step in Algorithm 3.

6.3 Algorithms and Implementation

6.3.1 API Reference

Five functions cover the full lifecycle of a ring group: registration, per-operation multiplication, chained accumulation, extra commitment inputs, and finalisation. The three subsections below trace how a concrete circuit, BN254 pairing verification, calls these: one shared skeleton, prover work running inline within it, verifier work deferred to its end.

`NewPolyRingCheck(mod, modEvalFn)` Registers a *ring group*: a modulus polynomial P paired with an optional small-coefficient evaluator `modEvalFn`. Returns a `*PolyRingGroupChecks` handle shared by every multiplication under that modulus; multiple groups with different moduli may coexist in the same circuit.

`MulPolyRings(group, A_1, ..., A_k)` Computes $R = (\prod_j A_j) \bmod P$ outside the circuit via `polyRingMulHint` (prover), range-checks R's limbs inside the circuit (verifier), and enqueues the tuple $(A_j; R; Q)$ in the group's deferred check list. Returns R immediately so callers can continue building the circuit.

`PolyRingAccumulator` A helper that accumulates a product chain $\prod_i F_i$ and flushes to `MulPolyRings` automatically when the accumulated degree would exceed a configurable bound `targetDeg`, avoiding excessive hint call overhead. `Mul` enqueues one factor; `Sqr` enqueues the current state twice; `Eval` forces an immediate flush and returns the remainder.

`ToCommit(e_1, ...)` Adds extra elements to the first commitment (the one that derives challenge z). Required for hinted values—such as the residue witness c in pairing checks—whose correctness is asserted *inside* the polynomial identity protocol itself, so they must be absorbed before z is sampled.

`performDeferredRingChecks` (internal) Called once by the gnark backend at circuit finalisation. Implements Equation (5) via two sequential `api.Commit` calls (Section 5.5.2), producing z then x, followed by one `AssertIsEqual` per group. No in-circuit multiplications over ring elements occur before this point beyond the remainder range-checks.

6.3.2 Shared Circuit Skeleton

Algorithms 3 and 4 walk the identical Miller loop structure, differing only in what happens at each step: the prover computes (Q_i, R_i), the verifier checks them. The implementation exploits this by using a single circuit function per curve, `sw_bn254/pairing.go` and `sw_bls12381/pairing.go`, that traces this shared skeleton once. At every step the code calls `MulPolyRings` (via `PolyRingAccumulator`) with the same operands both algorithms process. Prover's work is performed during the witness generation phase, while the verifier's work is performed by the circuit's constraints and commitment API (Section 5.5.2).

6.3.3 Prover's and Verifier's Work

Each `MulPolyRings` call triggers `polyRingMulHint` to prepare Q_i, R_i and queue the deferred check. This is Algorithm 3's work, executed inline during witness generation. The remainder R_i is returned to the caller so the circuit can continue building. Then `performDeferredRingChecks` calls `api.Commit` with all R_i to obtain the challenge z. Then it calls `quotientsRLCHint` to get Q_{acc}. After Q_{acc}, second commit absorbs z and Q_{acc} to produce x. Now verifiers work starts as described in Algorithm 4, Polynomials R_i and Q_{acc} are evaluated at x, and Equation (5) is verified using the Schwartz Zippel lemma with one `AssertIsEqual` call.

6.4 Usage

```
// Register the ring group once at initialisation
fp, _ := emulated.NewField[emulated.BN254Fp](api)
modPoly := fp.MakePoly(82, 0, 0, 0, 0, 0, -18, 0, 0, 0, 0, 0, 1)
ring := fp.NewPolyRingCheck(modPoly, nil)
// Operations on ring elements A, B, C
res, _ = fp.MulPolyRings(ring, A, B, C) // compute (A·B·C)
// Accumulate a product chain
acc := f.NewPolyRingAccumulator(ring, targetDeg)
acc.Mul(A).Mul(B).Sqr()  // enqueue (A·B)² without multiplying yet
rem := acc.Eval()        // multiply and return the remainder
```

6.5 Implementation Details and Performance

6.5.1 Schoolbook vs Horner's Method for Polynomial Evaluation

Horner's method for evaluations in `EvalPoly` across emulated elements causes a linear increase in the limbs size and count per coefficient. This necessitates reductions after each multiplication. Instead, we cache powers of the evaluation point x in $\mathcal{X} = (x^0, x^1, \ldots, x^d)$ and evaluate each polynomial as an inner product of $\mathcal{X}$ and the coefficient array $\mathbf{F}$:

$$\mathcal{X} \cdot \mathbf{F} = \mathcal{X}_0 \cdot F_0 + \mathcal{X}_1 \cdot F_1 + \cdots + \mathcal{X}_d \cdot F_d$$

In the emulated field context, Horner's method evaluations are replaced by this inner product with $\mathcal{X}$. This allows us to use the inner product optimization described below.

6.5.2 Native field operations for inner products

Modular reductions can be deferred or entirely elided for operations expressible as inner products, such as polynomial evaluations and random linear combinations. This optimization leverages the capacity headroom provided by representing small emulated limbs (e.g., 64-bit limbs) within a significantly larger native field element.

Deferred Reduction: Intermediate scalars are permitted to grow over the integers $\mathbb{Z}$ during limb-wise multiplications and linear accumulations.

Preservation of Identity: Algebraic identities are strictly preserved provided the maximum accumulated limb magnitude remains bounded below the native characteristic p.

Handling Asymmetry: In circuits with asymmetric multiplicative depths, operands may accumulate as disparate polynomial multiples of the emulated characteristic p_{emul}. Consequently, a terminal single-element reduction suffices for equivalence testing, bypassing the constraint overhead of intermediate normalization.

Lemma 1 (Overflow safety for deferred reduction)**.** *Let $\mathbb{F}_q$ be emulated with limbs of width b bits inside a native field $\mathbb{F}_p$. Consider evaluating a degree-d polynomial via an inner product over the cached power vector $\mathcal{X}$. Since limbs are accumulated independently, each result-limb is a sum of $d+1$ products of two b-bit values, so its magnitude is bounded by 2^β with $\beta = 2b + \lceil \log_2(d+1) \rceil$.*

The deferred single-element reduction is safe whenever $\beta < \log_2 p$.

Corollary 1 (BN254 pairing instantiation)**.** *For BN254 pairing verification: $b = 64$, $d \leq 256$, and $p \approx 2^{254}$. Then*

$$\beta \;=\; 2 \cdot 64 + \lceil \log_2 257 \rceil \;=\; 128 + 9 \;=\; 137 \;<\; 254,$$

so the deferred reduction is safe.

Beyond this static bound, the implementation tracks each element's overflow at compile time and `InnerProductNoReduce` inserts a reduction whenever an accumulation would exceed the native capacity, so Lemma 1 is also enforced dynamically for rings and degrees other than the BN254 instantiation above.

7 Conclusion

We have shown that choosing the right granularity for polynomial identity testing—entire Miller loop steps rather than individual field multiplications—yields substantial and measurable improvements in arithmetized pairing verification. The key technique, collapsing a composite multi-operand product into a single Euclidean division check with batched quotient accumulation, is simple to implement and broadly applicable to other multi-step cryptographic computations in arithmetized environments.

7.1 Applications and Impact

Our approach delivers maximum impact in resource-constrained execution environments where custom auxiliary data can be provided for cheaper verification. **Recursive proof systems** benefit the most, greatly reducing outer circuit complexity through fewer commitments and operations. **ZKVMs** (RISC Zero, SP1, Jolt) benefit naturally as they express computation as constraint systems compatible with our polynomial framework. **Calldata-constrained blockchains** (Ethereum L2s, BitVM) also benefit, as fewer commitments mean smaller proof transcripts and associated calldata costs.

This is already implemented in-production in Garaga [FE] for CairoVM and is currently the standard for all pairing based ZK proofs on Starknet. The polynomial ring

Table 6: Performance improvements in Garaga: Cairo implementation on Starknet

MULMOD		ADDMOD		COMMITS		VM STEPS	
Before	After	Before	After	Before	After	Before	After
BN254: 3-Pairing with Miller loop and final exponentiation							
19,142	12,656	21,686	15,142	5,689	2,807	338,678	212,902
33.9%		**30.2%**		**50.7%**		**37.1%**	
BLS12–381: 3-Pairing with Miller loop and final exponentiation							
16,484	11,287	20,669	15,420	5,421	3,167	325,757	219,155
31.5%		**25.4%**		**41.6%**		**32.7%**	

MULMOD/ADDMOD: Field operations in $\mathbb{F}_p$ via built-in function calls
COMMITS: Elements committed for Fiat-Shamir, see § 5.5.2
VM STEPS: CairoVM Steps [Sta22] (CPU Cycles)

toolkit (Section 6) in gnark, submitted as a part of this work, will directly impact the `evmprecompiles/ecpair` precompile via `AssertMillerLoopAndFinalExpIsOne`.

7.2 Implementation and Performance

The Garaga implementation [FE] in Cairo/Starknet (2024) sufficiently reduced the commitment overhead and modulo operations (Table 6) to fit Groth16 proof verification within Starknet's transaction limits for the first time. However, Cairo [Sta22] is a specialized environment with its own cost metrics, making direct comparison with mainstream ZK frameworks difficult. This work situates the same technique within gnark—the standard framework for field emulation in arithmetized circuits—providing a systematic benchmark against an industry baseline.

In Table 6, the *Before* column is Garaga verifying each extension field multiplication individually via the polynomial identity testing of [Fel23]; the *After* column is Garaga using our consolidated relations. MULMOD/ADDMOD count modular field operations, COMMITS count Poseidon [GKR+21] input commitments for Fiat-Shamir, and VM STEPS count CairoVM CPU cycles—together a low-level estimate of computational complexity inside the virtual machine.

Our polynomial ring toolkit implementation in gnark is benchmarked in Table 7, measured via a 3-pairing product check (see below) on BN254 and BLS12–381. We compare our results against a **gnark baseline** using the upstream `emulated` package [Cona], which represents the state-of-the-art for emulated pairings and includes the optimizations of [Hou23] and [NE24].

The **This Work** numbers were obtained by compiling the exact same benchmark against our fork, replacing the baseline's extension field multiplications with our new toolkit. No changes were made to the circuit definition or test harness. Both implementations exploit the sparseness of the Miller line functions (Section 4.1).

Benchmark circuit. Both implementations are evaluated against the same gnark circuit: a 3-pairing multi-pairing check $\prod_{i=1}^{3} e(P_i, Q_i) = 1$ via `PairingCheck`. This corresponds to the pairing workload of a Groth16 verification, which reduces to a product-of-pairings check over three variable pairs once the constant $e(\alpha, \beta)$ term from the trusted setup is folded in. The same 3-pairing configuration is used in the CairoVM benchmarks of Table 6, allowing a like-for-like comparison across environments.

Test circuit can be found at `sw_bn254` package in `g16_simulation_test.go` file in our implementation repo - https://anonymous.4open.science/r/tiny-gnark-DDCD/.

Table 7: Performance comparison: gnark implementation. Three-pairing product check on BN254 and BLS12–381

Metric	gnark Baseline	This Work	Reduction
BN254 – SCS (Plonkish) Constraint System			
Nb Constraints	1,592,440	1,035,550	**35.0%**
Nb $\mathbb{F}_q$ Committed	571,125	269,616	**52.8%**
Compile Time	615.04 ms	347.13 ms	**43.6%**
Compile Memory	1.33 GB	0.82 GB	**38.3%**
Solve Time	167.57 ms	97.30 ms	**41.9%**
Solve Memory	348.73 MB	181.24 MB	**48.0%**
BN254 – R1CS Constraint System			
Nb Constraints	491,447	316,167	**35.7%**
Nb $\mathbb{F}_q$ Committed	571,125	269,616	**52.8%**
Compile Time	617.63 ms	367.25 ms	**40.5%**
Compile Memory	1.60 GB	0.98 GB	**38.8%**
Solve Time	175.42 ms	106.36 ms	**39.4%**
Solve Memory	173.26 MB	112.72 MB	**34.9%**
BLS12–381 – SCS (Plonkish) Constraint System			
Nb Constraints	1,784,247	1,191,105	**33.2%**
Nb $\mathbb{F}_q$ Committed	660,160	300,104	**54.5%**
Compile Time	650.02 ms	410.48 ms	**36.9%**
Compile Memory	1.48 GB	0.97 GB	**34.5%**
Solve Time	180.05 ms	111.60 ms	**38.0%**
Solve Memory	362.14 MB	284.30 MB	**21.5%**
BLS12–381 – R1CS Constraint System			
Nb Constraints	545,510	368,434	**32.5%**
Nb $\mathbb{F}_q$ Committed	660,160	300,104	**54.5%**
Compile Time	686.95 ms	424.01 ms	**38.3%**
Compile Memory	1.74 GB	1.13 GB	**35.1%**
Solve Time	182.72 ms	121.15 ms	**33.7%**
Solve Memory	231.77 MB	118.61 MB	**48.8%**

Hardware: Apple M3 Pro (`goarch:arm64`), 36gb, macOS Tahoe 26.5.2

$\mathbb{F}_q$ Committed: Elements committed via `api.Commit` calls for Fiat-Shamir, see § 5.5.2

7.3 Further Work

Several natural extensions of this work remain open. First, our framework could be extended to additional curves beyond BN254 and BLS12–381, notably **BW6–761** for recursive proof chain verification on Ethereum.

A further direction is exploring the technique for native field pairings, such as BLS12–377 inside BW6–761 [Hou23] scalar fields, where the trade-off between batching savings and commitment costs warrants dedicated analysis.

References

[Ano24] Anonymous. Cairo pairing. https://anonymous.4open.science/r/cairo_pairing-83DA, 2024. Reference suppressed for double-blind review.

[Bel] Alexandre Belling. Multi-round, multi-commitment. https://hackmd.io/x8KsadW3RRyX7YTCFJIkHg.

[BGM+10] Jean-Luc Beuchat, Jorge Enrique González-Díaz, Shigeo Mitsunari, Eiji Okamoto, Francisco Rodríguez-Henríquez, and Tadanori Teruya. High-speed software implementation of the optimal ate pairing over barreto-naehrig curves. In *Pairing-Based Cryptography – Pairing 2010*, volume 6487 of *Lecture Notes in Computer Science*, pages 21–39. Springer, 2010.

[BGW+22] Dan Boneh, Sergey Gorbunov, Riad S. Wahby, Hoeteck Wee, Christopher A. Wood, and Zhenfei Zhang. BLS Signatures. Internet-draft, Internet Engineering Task Force, June 2022. Work in Progress.

[BSB23] Alexandre Belling, Azam Soleimanian, and Olivier Bégassat. Recursion over public-coin interactive proof systems; faster hash verification. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, CCS '23, pages 1422–1436, New York, NY, USA, 2023. Association for Computing Machinery.

[BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In *Theory of Cryptography – TCC 2016-B*, volume 9986 of *Lecture Notes in Computer Science*, pages 31–60. Springer, 2016.

[Cona] Consensys. gnark: A fast zk-snark library. https://github.com/Consensys/gnark.

[Conb] Consensys. gnark emulated fields. https://github.com/Consensys/gnark/tree/master/std/algebra/emulated.

[DHM04] Scott Vanstone Darrel Hankerson and Alfred Menezes. *Guide to Elliptic Curve Cryptography*. Springer-Verlag, 2004. Non-adjacent form representation.

[DL78] Richard A. Demillo and Richard J. Lipton. A probabilistic remark on algebraic program testing. *Information Processing Letters*, 7(4):193–195, 1978.

[FE] Feltroidprime and Keep Starknet Strange Exploration. Garaga: State-of-the-art elliptic curve tooling and snarks verification for cairo and starknet. https://github.com/keep-starknet-strange/garaga.

[Fel23] Feltroidprime. Faster extension field multiplications for emulated pairing circuits. https://hackmd.io/@feltroidprime/B1eyHHXNT, 2023. HackMD.

[FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology — CRYPTO' 86*, pages 186–194, Berlin, Heidelberg, 1987. Springer Berlin Heidelberg.

[GGPR13] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. Quadratic span programs and succinct nizks without pcps. In *Advances in Cryptology - EUROCRYPT 2013*, pages 626–645. Springer, 2013. Foundational QAP framework for R1CS.

[GKR+21] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. Poseidon: A new hash function for zero-knowledge proof systems. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 519–535. USENIX Association, 2021.

[GPR21] Lior Goldberg, Shahar Papini, and Michael Riabzev. Cairo – a turing-complete STARK-friendly CPU architecture. Cryptology ePrint Archive, Paper 2021/1063, 2021.

[Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology – EUROCRYPT 2016*, volume 9666 of *Lecture Notes in Computer Science*, pages 305–326. Springer, 2016.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. https://eprint.iacr.org/2019/953, 2019. Cryptology ePrint Archive, Paper 2019/953.

[Hou23] Youssef El Housni. Pairings in rank-1 constraint systems. In *Topics in Cryptology – CT-RSA 2023*, volume 13871 of *Lecture Notes in Computer Science*, pages 321–345. Springer, 2023.

[KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *Advances in Cryptology - ASIACRYPT 2010*, pages 177–194, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.

[NE24] Andrija Novakovic and Liam Eagen. On proving pairings. https://eprint.iacr.org/2024/640, 2024. Cryptology ePrint Archive, Paper 2024/640.

[Sch79] Jacob T. Schwartz. Probabilistic algorithms for verification of polynomial identities. In Edward W. Ng, editor, *Symbolic and Algebraic Computation*, pages 200–215, Berlin, Heidelberg, 1979. Springer Berlin Heidelberg.

[Sch80] J. T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *J. ACM*, 27(4):701–717, October 1980.

[Sta22] Starkware. Cairo: A turing-complete stark-friendly cpu architecture. https://www.cairo-lang.org/, 2022.

[Zip79] Richard Zippel. Probabilistic algorithms for sparse polynomials. In Edward W. Ng, editor, *Symbolic and Algebraic Computation*, pages 216–226, Berlin, Heidelberg, 1979. Springer Berlin Heidelberg.